

Lab 6: Testing ViewModel Property Change Notifications with NUnit

Goal

Learn how to write unit tests for `INotifyPropertyChanged` ViewModels using NUnit and a helper class. You'll test change notifications and build confidence in ViewModel behavior.

Step-by-Step Instructions

Step 1: Create the Solution and Projects

1. Open Visual Studio.
2. Create a solution named `ViewModelTestingLab`.
3. Add two projects:
 - `ViewModelTestingLab.Core` (Class Library)
 - `ViewModelTestingLab.Tests` (NUnit Test Project)
4. Add a reference from `ViewModelTestingLab.Tests` to `ViewModelTestingLab.Core`.

Step 2: Implement the ViewModel Class

In the `core` project, create a `PersonViewModel`:

```
using System.ComponentModel;

public class PersonViewModel : INotifyPropertyChanged
{
    private string _name;
    private int _age;

    public string Name
    {
        get => _name;
        set
        {
            if (_name != value)
            {
                _name = value;
                OnPropertyChanged(nameof(Name));
            }
        }
    }
}
```

```

    }
}

public int Age
{
    get => _age;
    set
    {
        if (_age != value)
        {
            _age = value;
            OnPropertyChanged(nameof(Age));
            OnPropertyChanged(nameof(IsAdult));
        }
    }
}

public bool IsAdult => Age >= 18;

public event PropertyChangedEventHandler? PropertyChanged;

protected void OnPropertyChanged(string propertyName)
    => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
}

```

Step 3: Add the `NotifyPropertyChangedTester` Class

In the `Tests` project, add this helper class (from the instructor-provided file):

```

// NotifyPropertyChangedTester.cs
using System.ComponentModel;
using NUnit.Framework;

public class NotifyPropertyChangedTester
{
    public NotifyPropertyChangedTester(INotifyPropertyChanged viewModel)
    {
        if (viewModel == null)
            throw new ArgumentNullException(nameof(viewModel));

        Changes = new List<string>();
        viewModel.PropertyChanged += OnPropertyChangedEvent;
    }

    private void OnPropertyChangedEvent(object? sender, PropertyChangedEventArgs e)
    {
        if (string.IsNullOrEmpty(e.PropertyName))

```

```

        throw new InvalidOperationException("PropertyName was null or empty");

        Changes.Add(e.PropertyName);
    }

    public List<string> Changes { get; private set; }

    public void AssertChange(int index, string expectedProperty)
    {
        Assert.That(Changes, Is.Not.Null);
        Assert.That(index < Changes.Count, $"Expected at least {index + 1} changes.");
        Assert.That(Changes[index], Is.EqualTo(expectedProperty));
    }

    public void AssertChange(string expectedProperty)
    {
        Assert.That(Changes, Does.Contain(expectedProperty),
            $"Expected a change notification for '{expectedProperty}'.");
    }
}

```

Step 4: Write NUnit Tests

Create a test class in `Tests` project:

```

[TestFixture]
public class PersonViewModelTests
{
    [Test]
    public void ChangingName_ShouldRaisePropertyChanged()
    {
        var viewModel = new PersonViewModel();
        var tester = new NotifyPropertyChangedTester(viewModel);

        viewModel.Name = "Alice";

        tester.AssertChange("Name");
    }

    [Test]
    public void ChangingAge_ShouldRaisePropertyChangedForAgeAndIsAdult()
    {
        var viewModel = new PersonViewModel();
        var tester = new NotifyPropertyChangedTester(viewModel);

        viewModel.Age = 25;
    }
}

```

```
tester.AssertChange(0, "Age");  
tester.AssertChange(1, "IsAdult");  
}  
}
```

✅ Step 5: Run the Tests

- Build the solution.
 - Run the tests using the Test Explorer.
 - Confirm all tests pass.
-

✅ What You Learned

- How to implement `INotifyPropertyChanged` correctly in ViewModels
 - How to use a test helper to assert property change notifications
 - How to write readable, reliable unit tests for UI logic
-

✍️ Wrap-Up Challenge

Add another ViewModel property (e.g., `Email`) and test its notification. Bonus: Add validation logic and test that it doesn't raise events when the value hasn't changed.